

Politechnika Świętokrzyska Wydział Elektrotechniki, Automatyki i Informatyki	
Modelowanie i Analiza Systemów Informatycznych – Projekt	
Temat: System rejestracji czasu pracy	Zespół: Patryk Kręgiel Szymon Kuzio grupa 1ID24B

1. Cel i zakres systemu

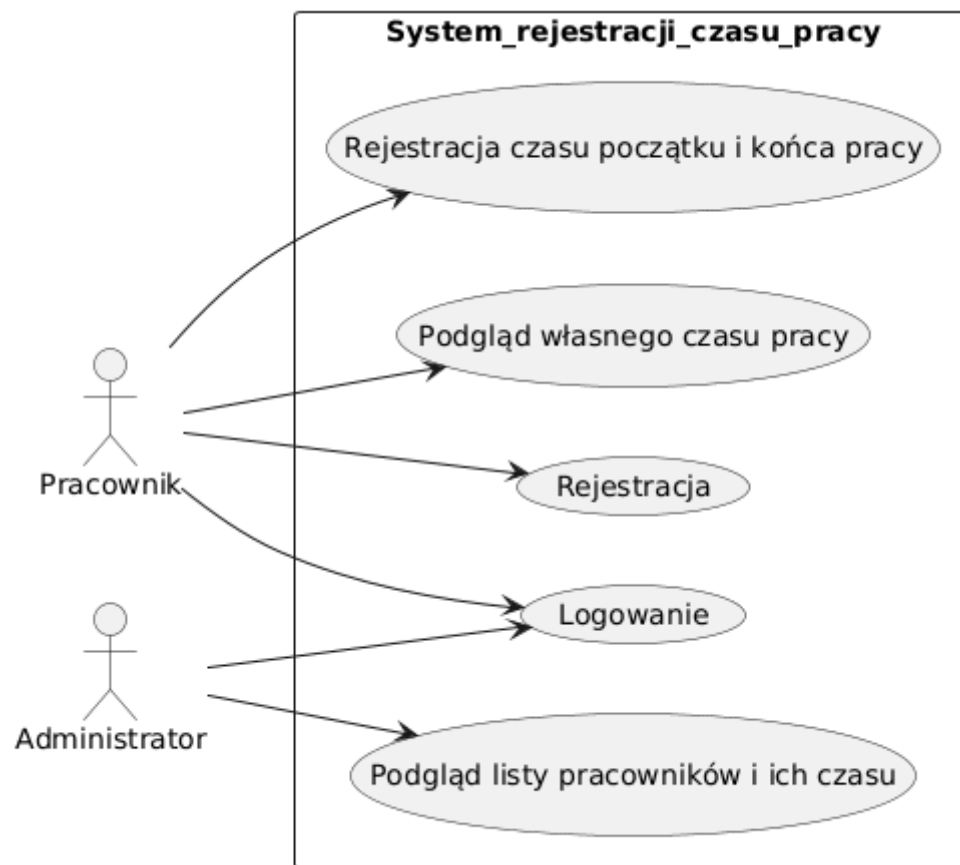
Celem systemu jest implementacja aplikacji umożliwiającej rejestrację czasu pracy. Ma ona umożliwiać zapis godziny rozpoczęcia i zakończenia pracy, a także przechowywać dane w bazie danych w sposób bezpieczny. Zarządzanie dostępem ma odbywać się poprzez system logowania.

System ma obejmować rejestrację nowych użytkowników oraz ich logowanie, monitorowanie czasu pracy pracowników, a także panel administratora wykorzystywany do przeglądu danych. Ponadto, system ma wykorzystywać bazę danych MySQL oraz mechanizm bezpiecznego przechowywania haseł.

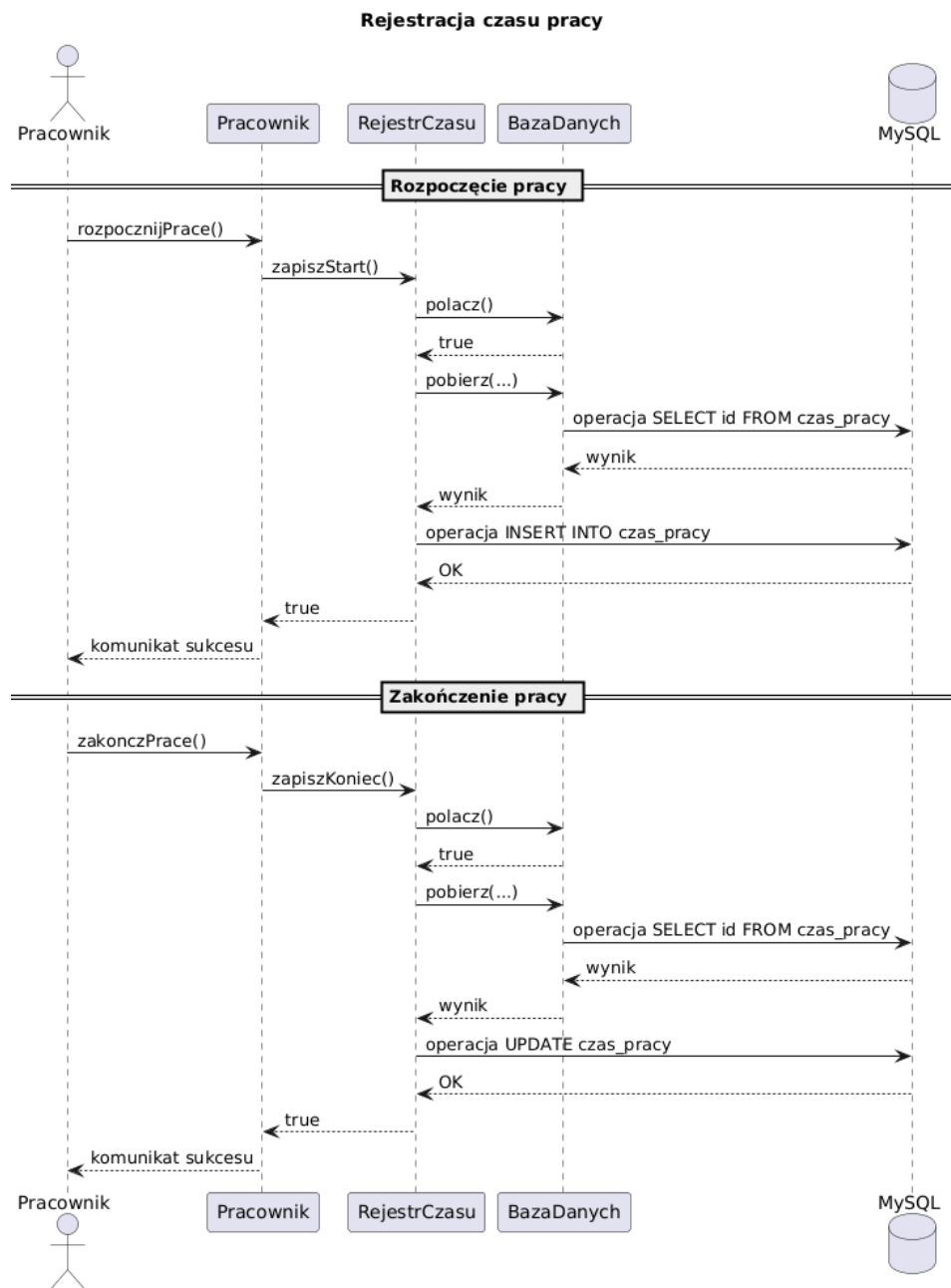
2. Aktorzy systemu

System będzie posiadał dwóch aktorów. Pierwszym aktorem jest pracownik, który będzie miał możliwość rejestracji i logowania do systemu, a także rejestracji czasu początku i końca pracy oraz podglądu własnego czasu pracy. Drugim aktorem jest administrator, który po zalogowaniu się będzie miał możliwość przeglądu listy pracowników oraz ich czasu pracy.

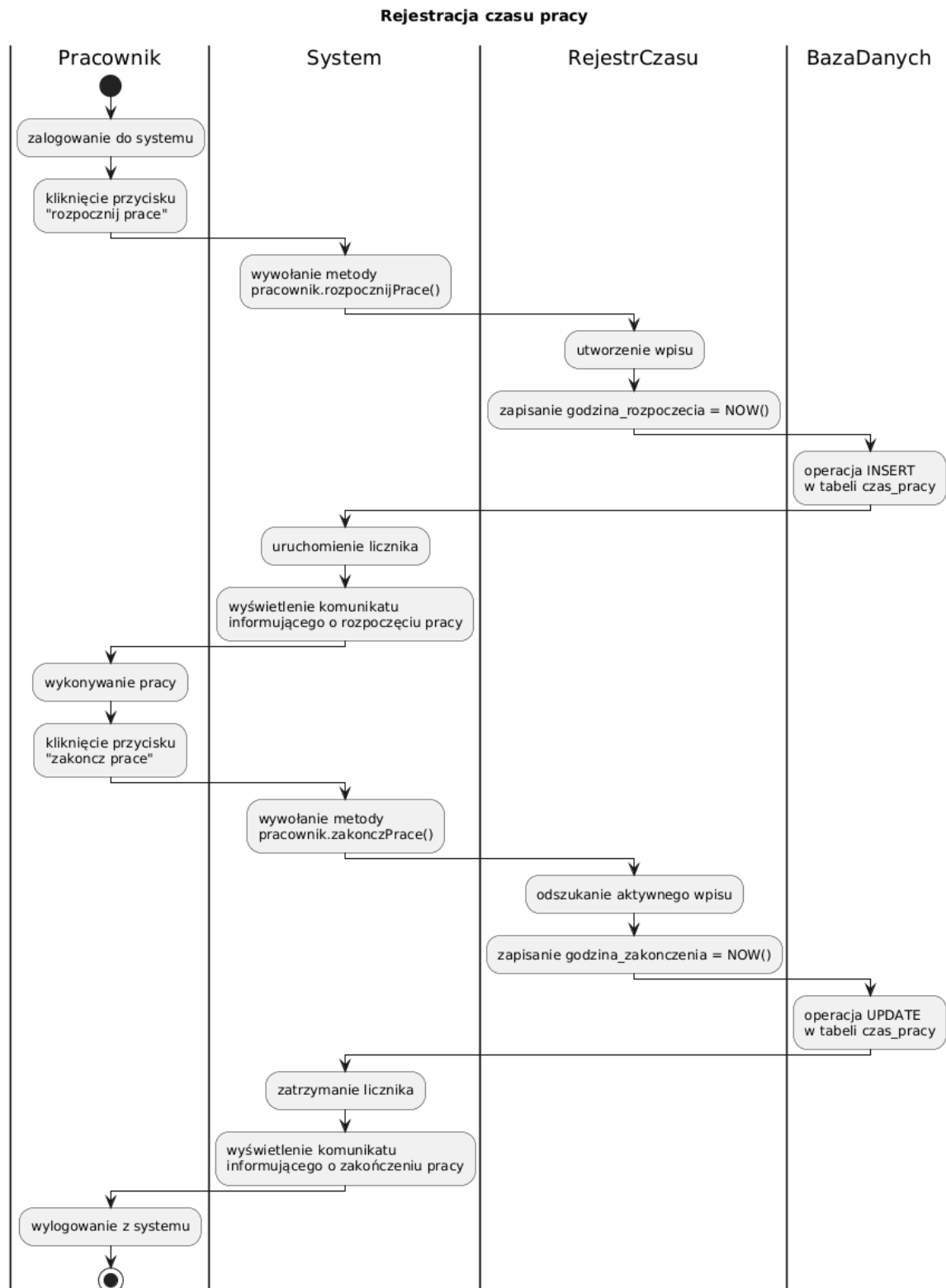
3. Diagram przypadków użycia



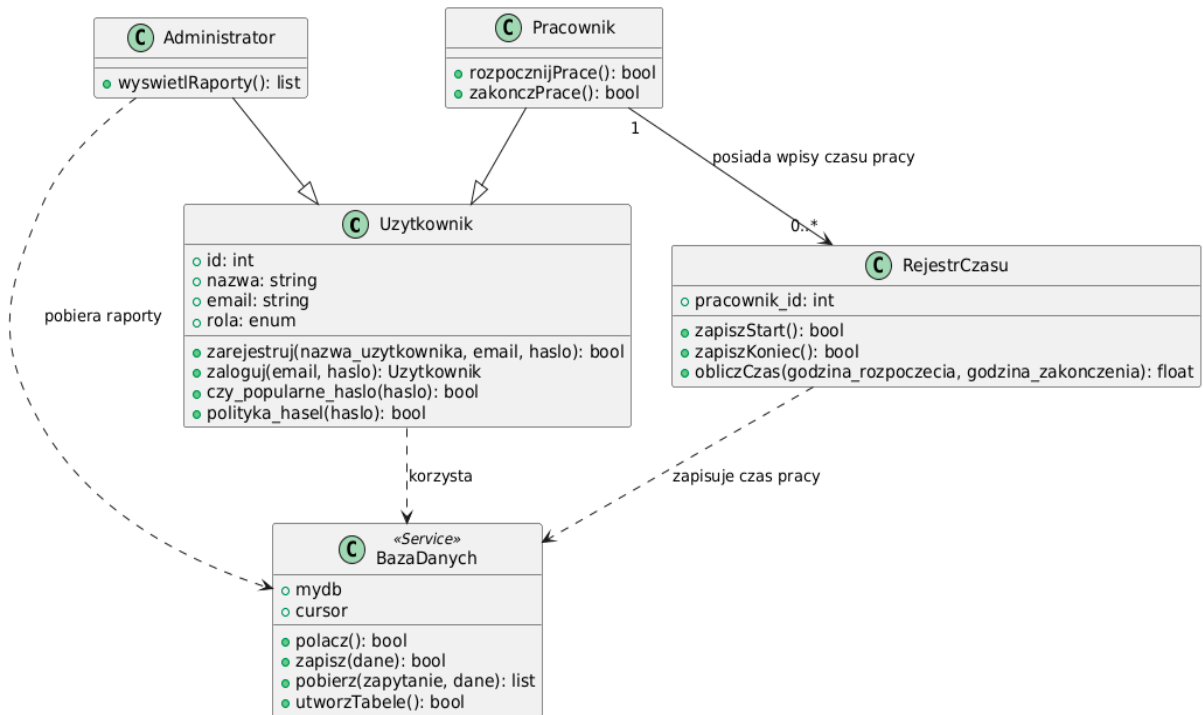
4. Diagram sekwencji



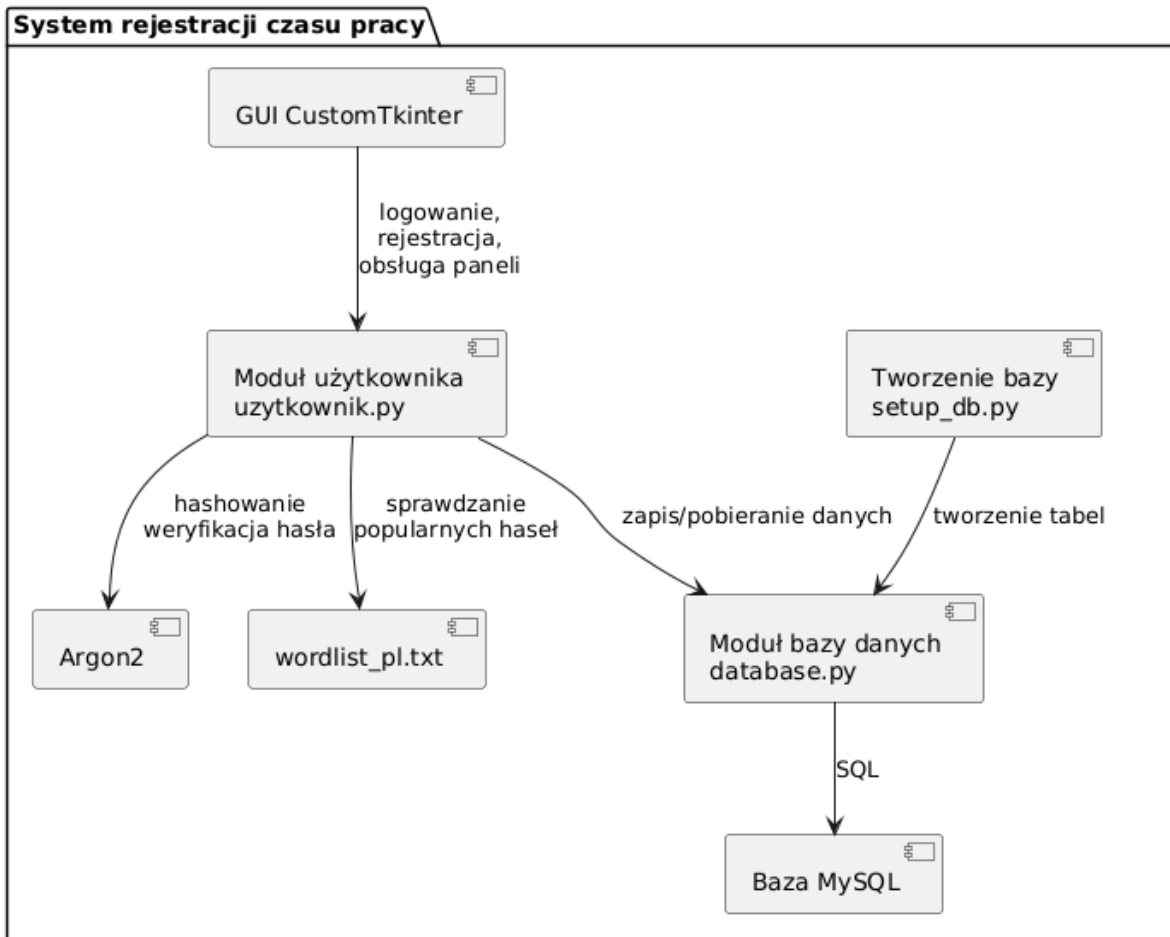
5. Diagram aktywności



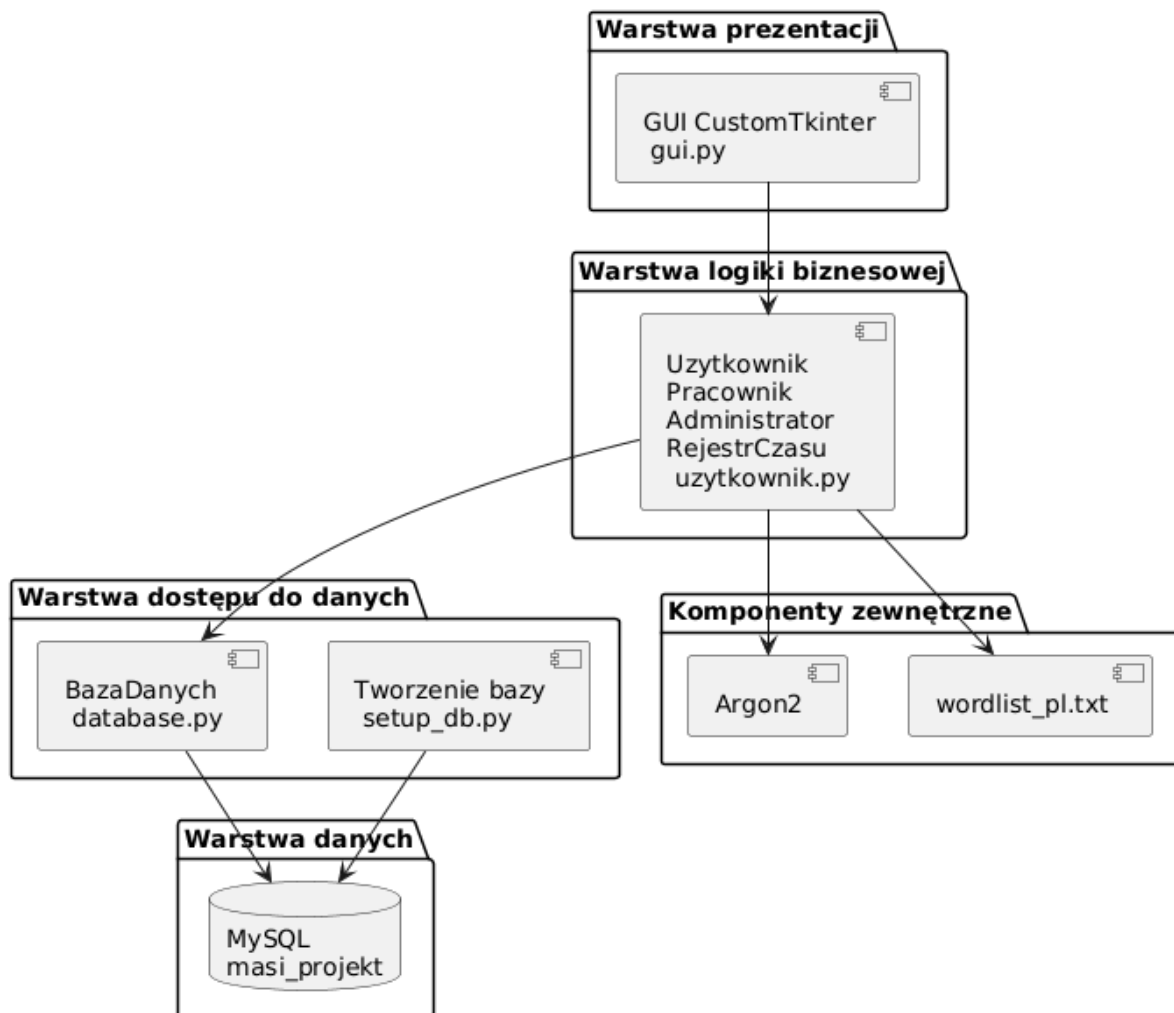
6. Diagram klas



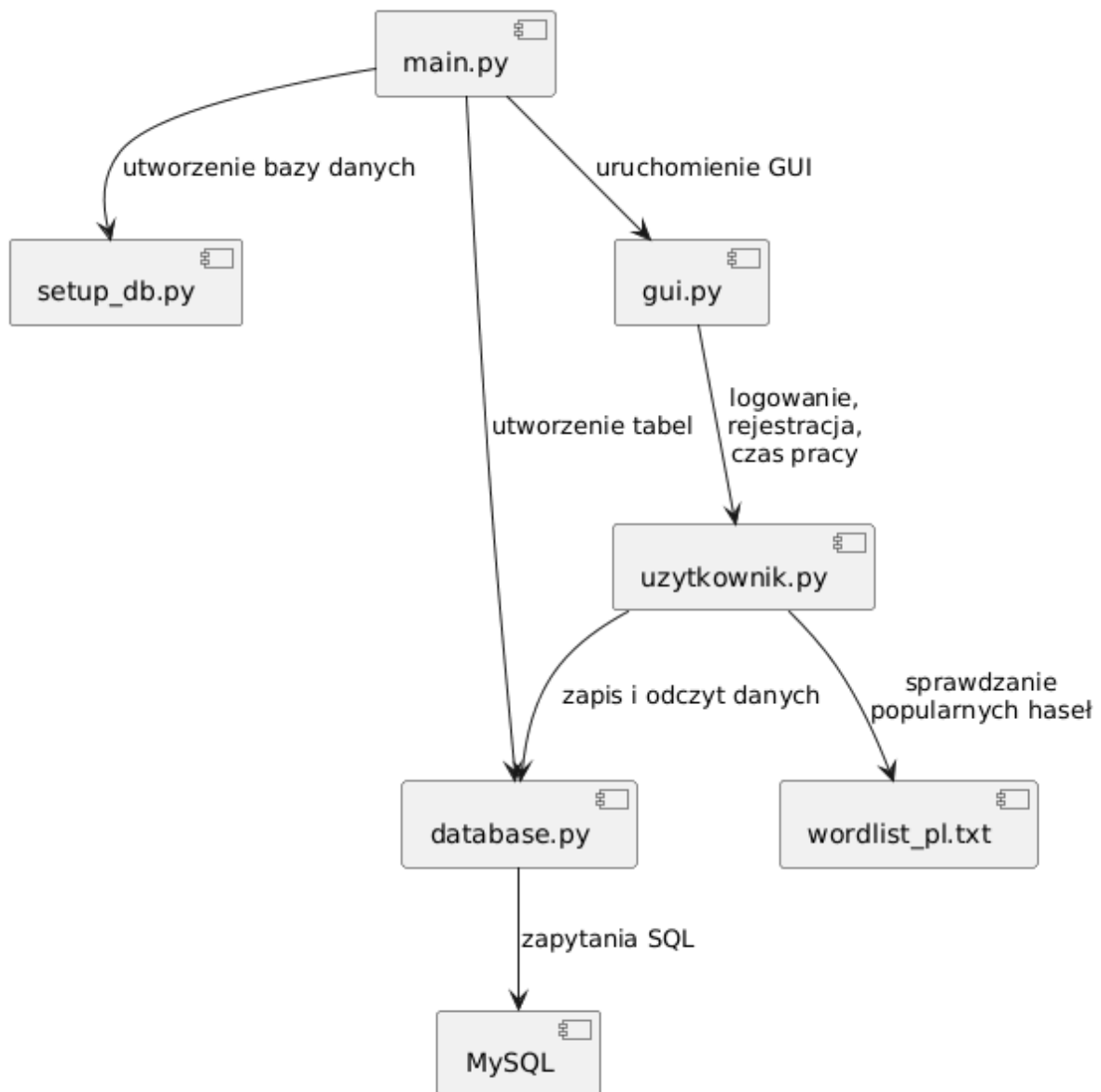
7. Diagram komponentów



8. Diagram warstw systemu



9. Diagram zależności między modułami



10. Fragmenty modelu analitycznego

Wymagania funkcjonalne:

- program ma umożliwiać rejestrację nowych pracowników,
- program ma umożliwiać logowanie użytkowników,
- program ma umożliwiać rejestrację czasu rozpoczęcia i zakończenia pracy,
- program ma umożliwiać utworzenie bazy danych MySQL podczas pierwszego uruchomienia,
- program ma umożliwiać zapisanie danych do bazy danych,
- program ma umożliwiać przeglądanie danych przez Administratora.

Wymagania niefunkcjonalne:

- użyteczność - program ma być prosty w obsłudze
- niezawodność - program ma działać niezawodnie
- wydajność - program ma szybko wykonywać działania
- bezpieczeństwo - program ma przechowywać hasła w sposób bezpieczny

11. Metryki kodu

Metryka	Wartość
Liczba klas	5
Liczba metod	17
Liczba modułów	5

11.1. Liczba klas

a) database.py:

-BazaDanych

b) uzytkownik.py:

-Uzytkownik

-Pracownik

-Administrator

-RejestrCzasu

11.2. Liczba metod

a) BazaDanych:

- __init__()
- polacz()
- zapisz()
- pobierz()
- utworzTabele()

b) Uzytkownik:

- __init__()
- zarejestruj()
- zaloguj()
- czy_popularne_haslo()
- polityka_hasel()

c) Pracownik:

- rozpocznijPrace()
- zakonczPrace()

d) Administrator:

- wyswietlRaporty()

e) RejestrCzasu:

- __init__()
- zapiszStart()
- zapiszKoniec()
- obliczCzas()

11.3. Liczba modułów

a) setup_db.py

b) database.py

c) uzytkownik.py

d) gui.py

e) main.py

12. Struktura programu

Aplikacja została zbudowana w sposób modularny, gdzie każdy moduł realizuje następujące czynności:

- moduł `setup_db.py` odpowiada za utworzenie bazy danych, jeśli nie istnieje.
- moduł `database.py` odpowiada za połączenie z bazą danych, utworzenie tabel, jeśli nie istnieją, zapis i pobieranie danych oraz operacje SQL.
- moduł `uzytkownik.py` odpowiada za rejestrację i logowanie użytkowników, politykę haseł, rozpoczęcie i zakończenie pracy oraz generowanie raportów.
- moduł `gui.py` odpowiada za utworzenie interfejsu graficznego.
- moduł `main.py` odpowiada za uruchomienie aplikacji.

12.1. Moduł `setup_db.py`

Moduł `setup_db.py` odpowiada za utworzenie bazy danych podczas pierwszego uruchomienia aplikacji. Odpowiada za to funkcja `create_db()`:

```
db = "masi_projekt"

def create_db():
    mydb = mysql.connector.connect(
        host="localhost",
        user="root",
        password=""
    )
    cursor = mydb.cursor()
    cursor.execute(f"CREATE DATABASE IF NOT EXISTS {db}")
    print("utworzono baze danych")
```

Powyższy kod realizuje nawiązanie połączenia z serwerem MySQL, a następnie tworzy kursor wykonujący polecenie `CREATE DATABASE`. Polecenie to wykonywane jest podczas pierwszego uruchomienia aplikacji. Informacja o utworzeniu bazy danych wyświetlona zostaje w konsoli.

12.2. Moduł database.py

Moduł database.py odpowiada za połączenie z bazą danych oraz jej obsługę. Za połączenie z bazą danych odpowiada funkcja `polacz()`:

```
def polacz(self):
    self.mydb = mysql.connector.connect(
        host = "localhost",
        user = "root",
        password = "",
        database = setup_db.db
    )

    self.cursor = self.mydb.cursor()
    return True
```

Powyższy kod nawiązuje połączenie z bazą danych, wykorzystując zdefiniowane parametry - nazwę hosta, nazwę użytkownika, hasło oraz nazwę bazy danych. Następnie tworzony jest kursor umożliwiający wykonywanie poleceń SQL. Zwracana przez funkcję wartość `True` informuje o utworzeniu połączenia.

Drugą funkcją odpowiedzialną za obsługę bazy danych jest funkcja `zapisz()`, która dodaje nowych użytkowników do tabeli `Uzytkownicy`:

```
def zapisz(self, dane):
    sql = """
        INSERT INTO Uzytkownicy (nazwa_uzytkownika, email, zahashowane_haslo,
salt)
        VALUES (%s, %s, %s, %s)"""
    try:
        self.cursor.execute(sql, dane)
        self.mydb.commit()
        return True
    except Exception as e:
        print(e)
        return False
```

Powyższy kod przygotowuje polecenie `INSERT INTO`, które jest zapytaniem sparametryzowanym - dzięki temu aplikacja zabezpieczona jest przed atakami SQL Injection. Po przygotowaniu zapytania, następuje jego wykonanie poprzez kursor i metodę `execute()`. W przypadku poprawnego wykonania zapytania, zwracana jest wartość `True` - w przeciwnym razie w konsoli zostaje wyświetlony błąd, a zwrócona zostaje wartość `False`.

Za odczyt danych z bazy odpowiada metoda pobierz():

```
def pobierz(self, zapytanie, dane=None):
    if dane is None:
        self.cursor.execute(zapytanie)
    else:
        self.cursor.execute(zapytanie, dane)
    return self.cursor.fetchall()
```

Powyższy kod realizuje odczyt danych z bazy. W przypadku braku parametrów, wykonywane jest zapytanie `SELECT *`, z kolei w przypadku wystąpienia parametrów wykonywane jest zapytanie sparametryzowane - jest to zabezpieczenie przed atakami SQL Injection. Następnie metoda `fetchall()` pobiera wszystkie rekordy i zwraca je w formie krotek.

Podczas pierwszego uruchomienia aplikacji, tworzone są tabele `Uzytkownicy` i `Czas_pracy`. Odpowiada za to funkcja `utworzTabele()`:

```
def utworzTabele(self):
    uzytkownicy_sql = """
    CREATE TABLE IF NOT EXISTS Uzytkownicy
    (id INT PRIMARY KEY AUTO_INCREMENT,
    nazwa_uzytkownika VARCHAR(255),
    email VARCHAR(255) UNIQUE,
    zahashowane_haslo VARCHAR(255),
    salt VARCHAR(255),
    rola ENUM('pracownik', 'administrator') DEFAULT 'pracownik')
    """

    self.cursor.execute(uzytkownicy_sql)

    czas_pracy_sql = """
    CREATE TABLE IF NOT EXISTS Czas_pracy
    (id INT PRIMARY KEY AUTO_INCREMENT,
    pracownik_id INT,
    godzina_roz poczeczia DATETIME,
    godzina_zakonczczenia DATETIME,
    FOREIGN KEY (pracownik_id) REFERENCES Uzytkownicy(id))
    """

    self.cursor.execute(czas_pracy_sql)

    self.mydb.commit()
    print("utworzono tabele Uzytkownicy")
    print("utworzono tabele Czas_pracy")
    return True
```

Powyższy kod wykonuje polecenia `CREATE TABLE` dla tabel zawierających informacje o użytkownikach oraz informacje o czasie pracy poszczególnych pracowników. W przypadku tabeli `Uzytkownicy`, domyślną rolą jest rola "pracownik". Tabele połączone są relacją "jeden do wielu" - jeden pracownik może odnotować wiele wpisów w tabeli `Czas_pracy`. Tabele tworzone są wyłącznie podczas pierwszego uruchomienia aplikacji, a informacja o utworzeniu tabel wyświetlona zostaje w konsoli.

12.3. Moduł uzytkownik.py

Moduł uzytkownik.py odpowiada za główną logikę opracowanego systemu. Za pomocą funkcji zarejestruj(), uzytkownik ma możliwość utworzenia konta:

```
password_hasher = argon2.PasswordHasher(memory_cost=12288, time_cost=3,
parallelism=1)

def zarejestruj(self, nazwa_uzytkownika, email, haslo):
    wynik_polityki = self.polityka_hasel(haslo)
    if wynik_polityki is not True:
        return wynik_polityki

    salt = secrets.token_hex(4)
    haslo_z_sola = haslo+salt
    zahashowane_haslo = password_hasher.hash(haslo_z_sola)

    dane = (nazwa_uzytkownika, email, zahashowane_haslo, salt)
    baza = database.BazaDanych()
    if baza.polacz():
        if baza.zapisz(dane):
            print(f"zarejestrowano uzytkownika {nazwa_uzytkownika}!\n")
            return True
        else:
            return "konto o podanym adresie e-mail juz istnieje!"
    return False
```

Powyższy kod odpowiada za weryfikację zgodności hasła z ustaloną polityką bezpieczeństwa, a następnie generuje 32-bitową sól. Dodawana jest ona do wprowadzonego przez uzytkownika hasła, a wynikiem końcowym jest hash utworzony przy pomocy algorytmu Argon2. Dane następnie zapisywane są do bazy danych - jeśli istnieje już konto o podanym adresie e-mail, wyświetlany jest stosowny komunikat. W przeciwnym razie uzytkownik zostaje dodany do bazy, a odpowiednia informacja zostaje wyświetlona w konsoli.

Za pomocą funkcji zaloguj(), uzytkownik ma możliwość zalogowania się na swoje konto:

```
def zaloguj(self, email, haslo):
    baza = database.BazaDanych()
    if not baza.polacz():
        return False

    sql = """
        SELECT id, nazwa_uzytkownika, email, zahashowane_haslo, salt, rola
        FROM Uzytkownicy
        WHERE email = %s"""

    wynik = baza.pobierz(sql, (email,))

    if len(wynik) == 0:
        print("nieprawidlowy e-mail lub haslo!\n")
        return False

    user = wynik[0]
```

```

id = user[0]
nazwa_uzytkownika = user[1]
email = user[2]
zahashowane_haslo = user[3]
salt = user[4]
rola=user[5]

haslo_z_sola = haslo+salt

try:
    password_hasher.verify(zahashowane_haslo, haslo_z_sola)
    if rola == "pracownik":
        return Pracownik(id, nazwa_uzytkownika, email, rola)
    if rola == "administrator":
        return Administrator(id, nazwa_uzytkownika, email, rola)
except:
    return False

```

Powyższy kod zestawia połączenie z bazą danych, a następnie przygotowuje zapytanie wyszukujące użytkowników o danym adresie e-mail. W przypadku użytkownika w bazie, wyświetlany jest stosowny komunikat. W przeciwnym wypadku pobierany jest rekord zawierający informacje o użytkowniku: identyfikator, login, adres e-mail, zahashowane hasło, sól oraz rolę. Następnie, na podstawie wprowadzonego hasła i pobranej soli, hasło zostaje odtworzone i zweryfikowane z wartością w bazie danych. Jeśli użytkownik ma przypisaną rolę "pracownik" lub "administrator", tworzony jest stosowny obiekt klasy.

Za politykę bezpieczeństwa haseł odpowiadają dwie funkcje - czy_popularne_haslo() oraz polityka_hasel():

```

def czy_popularne_haslo(self, haslo):
    with open("wordlist_pl.txt", 'r', encoding="utf-8") as file:
        popularne_hasla = file.read().splitlines()
        return haslo.lower() in [h.lower() for h in popularne_hasla]

def polityka_hasel(self, haslo):
    if len(haslo)<12:
        return "haslo musi miec minimum 12 znakow!"
    if len(haslo)>128:
        return "haslo nie moze miec wiecej niz 128 znakow!"
    if self.czy_popularne_haslo(haslo):
        return "zbyt popularne haslo!"
    return True

```

Powyższy kod realizuje dwie funkcjonalności zapewniające zgodność hasła z polityką bezpieczeństwa - sprawdzenie, czy hasło nie jest popularne, oraz czy spełnia wymagania odnośnie długości. Weryfikacja popularności hasła zrealizowana została poprzez odczyt pliku zawierającego listę popularnych haseł oraz sprawdzenie, czy wprowadzone przez użytkownika hasło znajduje się na liście. Kontrola podlega również długość hasła - musi mieć minimum 12 znaków i nie więcej niż 128 znaków.

W skład modułu uzytkownik.py wchodzi dwie klasy dziedziczące po klasie Uzytkownik - Pracownik oraz Administrator. W skład klasy Pracownik wchodzi dwie metody - rozpocznijPrace() oraz zakonczPrace():

```
class Pracownik(Uzytkownik):
    def rozpocznijPrace(self):
        rejestr = RejestrCzasu(self.id)
        return rejestr.zapiszStart()

    def zakonczPrace(self):
        rejestr = RejestrCzasu(self.id)
        return rejestr.zapiszKoniec()
```

W ramach metody rozpocznijPrace() tworzony jest nowy obiekt klasy RejestrCzasu, a następnie wywoływana jest metoda zapiszStart(). W przypadku zakończenia pracy, tworzony jest obiekt klasy RejestrCzasu oraz wywoływana jest metoda zapiszKoniec().

W skład klasy Administrator wchodzi jedna metoda - wyswietlRaporty():

```
class Administrator(Uzytkownik):
    def wyswietlRaporty(self):
        baza = database.BazaDanych()
        if not baza.polacz():
            return []

        sql_pracownicy = """
        SELECT id, nazwa_uzytkownika, email
        FROM Uzytkownicy
        WHERE rola = %s
        """

        pracownicy = baza.pobierz(sql_pracownicy, ("pracownik",))
        raport = []

        for pracownik in pracownicy:
            pracownik_id = pracownik[0]
            nazwa = pracownik[1]
            email = pracownik[2]

            sql_czas_pracy = """
            SELECT godzina_rozpoczecia, godzina_zakonczenia
            FROM czas_pracy
            WHERE pracownik_id = %s
            ORDER BY id DESC
            """

            czas_pracy = baza.pobierz(sql_czas_pracy, (pracownik_id,))
            raport.append((pracownik_id, nazwa, email, czas_pracy))

        return raport
```

Powyższy kod odpowiada za utworzenie i wyświetlenie raportów odnośnie czasu pracy pracowników. Pierwszym etapem tworzenia raportu jest połączenie z bazą danych, a następnie wykonywane jest pobranie listy wszystkich pracowników. Kolejnym krokiem jest przetworzenie danych o każdym z pracowników: identyfikatora,

nazwy oraz adresu e-mail. Następnie pobierana jest godzina rozpoczęcia i zakończenia pracy każdego pracownika. Tworzenie raportu kończy się dodaniem pobranych wpisów do raportu oraz jego zwrócenie.

Za obsługę rejestracji czasu pracy pracowników odpowiada klasa RejestrCzasu. W jej skład wchodzi cztery metody - konstruktor klasy oraz metody: zapiszStart(), zapiszKoniec() i obliczCzas():

```
class RejestrCzasu:
    def __init__(self, pracownik_id):
        self.pracownik_id = pracownik_id

    def zapiszStart(self):
        baza = database.BazaDanych()
        if not baza.polacz():
            return False

        sql_aktywnosc_pracownika = """
            SELECT id
            FROM czas_pracy
            WHERE pracownik_id = %s AND godzina_zakonczenia IS NULL
            ORDER BY id DESC
            LIMIT 1
        """

        aktywnosc_pracownika = baza.pobierz(sql_aktywnosc_pracownika,
            (self.pracownik_id,))

        if len(aktywnosc_pracownika) > 0:
            return "rozpoczeto prace!"

        sql_rozpoczecie_pracy = """
            INSERT INTO czas_pracy (pracownik_id, godzina_rozpoczecia,
            godzina_zakonczenia)
            VALUES (%s, NOW(), NULL)
        """

        baza.cursor.execute(sql_rozpoczecie_pracy, (self.pracownik_id,))
        baza.mydb.commit()
        return True

    def zapiszKoniec(self):
        baza = database.BazaDanych()
        if not baza.polacz():
            return False

        sql_aktywnosc_pracownika = """
            SELECT id
            FROM czas_pracy
            WHERE pracownik_id = %s AND godzina_zakonczenia IS NULL
            ORDER BY id DESC
            LIMIT 1
        """

        aktywnosc_pracownika = baza.pobierz(sql_aktywnosc_pracownika,
            (self.pracownik_id,))

        if len(aktywnosc_pracownika) == 0:
            return "rozpocznij prace!"
```

```

        id_rejestru = aktywnosc_pracownika[0][0]

        sql_zakonczenie_pracy = """
            UPDATE czas_pracy SET godzina_zakonczenia = NOW()
            WHERE id = %s
        """

        baza.cursor.execute(sql_zakonczenie_pracy, (id_rejestru,))
        baza.mydb.commit()
        return True

    def obliczCzas(self, godzina_rozpoczecia, godzina_zakonczenia):
        if godzina_rozpoczecia is None or godzina_zakonczenia is None:
            return "00:00:00"
        roznica = godzina_zakonczenia - godzina_rozpoczecia
        sekundy = int(roznica.total_seconds())

        h = sekundy // 3600
        m = (sekundy%3600) // 60
        s = sekundy % 60
        return f"{h:02}:{m:02}:{s:02}"

```

Metoda `__init__()` jest konstruktorem klasy - podczas tworzenia nowego obiektu, zapisywany jest identyfikator użytkownika. Metoda `zapiszStart()` odpowiada za zapis godziny rozpoczęcia pracy. Pierwszym etapem jest nawiązanie połączenia z bazą danych. Następnie wykonywane jest zapytanie `SELECT` sprawdzające aktywność pracownika - jeśli w bazie danych nie ma wpisów zawierających godzinę rozpoczęcia pracy, za pomocą polecenia `INSERT INTO` dodawany jest nowy wpis aktywności. Metoda `zapiszKoniec()` odpowiada za zapis godziny zakończenia pracy. Po połączeniu z bazą danych, wykonywane jest zapytanie `SELECT` sprawdzające aktywność pracownika. Następnie pobierany jest identyfikator wpisu, a następnie wykonywane jest polecenie `INSERT INTO`, za pomocą którego w bazie zostaje umieszczona godzina zakończenia pracy. Metoda `obliczCzas()` służy do obliczenia łącznego czasu pracy w formacie HH:mm:ss. Jeśli dany użytkownik nie rozpoczął lub nie zakończył pracy, zwracana jest wartość równa zero. W przeciwnym wypadku obliczana jest różnica na podstawie godziny zakończenia i rozpoczęcia pracy. Wynik zamieniany jest na sekundy, a następnie konwertowany do formatu HH:mm:ss.

12.4. Moduł gui.py

Moduł gui.py odpowiada za utworzenie i wyświetlanie interfejsu graficznego aplikacji. W skład GUI wchodzi okna: rejestracji, logowania oraz panel pracownika i administratora. W skład panelu pracownika wchodzi trzy funkcje: aktualizuj_licznik(), rozpocznij_prace() oraz zakoncz_prace():

```
def okno_pracownik(pracownik):
    wyczysc_okno()

    label_nazwa_pracownika = ctk.CTkLabel(root, text=f"panel pracownika:
{pracownik.nazwa}", font=("Arial", 20))
    label_nazwa_pracownika.pack(pady=(75, 10))

    root.czas_pracy = 0
    root.licznik = False
    label_czas_pracy = ctk.CTkLabel(root, text="00:00:00", font=("Arial", 40))
    label_czas_pracy.pack(pady=(50, 10))

    def aktualizuj_licznik():
        if root.licznik:
            root.czas_pracy += 1

            h = root.czas_pracy // 3600
            m = (root.czas_pracy%3600) // 60
            s = root.czas_pracy % 60
            label_czas_pracy.configure(text=f"{h:02}:{m:02}:{s:02}")
            root.after(1000, aktualizuj_licznik)

    def rozpocznij_prace():
        wynik = pracownik.rozpocznijPrace()

        if wynik is True:
            root.czas_pracy = 0
            root.licznik = True
            label_czas_pracy.configure(text="00:00:00")
            root.after(1000, aktualizuj_licznik)
            button_wyloguj.configure(state="disabled", fg_color="#686868")
            messagebox.showinfo("sukces", "rozpoczeto prace!")
        else:
            messagebox.showinfo("blad", wynik)

    def zakoncz_prace():
        wynik = pracownik.zakonczPrace()

        if wynik is True:
            root.licznik = False
            button_wyloguj.configure(state="normal", fg_color="white")
            messagebox.showinfo("sukces", "zakonczono prace!")
        else:
            messagebox.showinfo("blad", wynik)

    button_start = ctk.CTkButton(root, text="rozpocznij prace",
command=rozpocznij_prace, **BUTTON)
    button_start.pack(pady=(50, 25))

    button_stop = ctk.CTkButton(root, text="zakoncz prace",
```

```
command=zakonczz_prace, **BUTTON)
    button_stop.pack(pady=25)

    button_wyloguj = ctk.CTkButton(root, text="wyloguj", command=okno_menu,
**BUTTON)
    button_wyloguj.pack(pady=25)
```

Powyższy kod usuwa wszystkie elementy poprzednio wyświetlanego okna, a następnie tworzy etykiety służące do wyświetlania nazwy zalogowanego użytkownika oraz licznika odmierzający czas pracy. Za pomocą funkcji aktualizuj_licznik() następuje zmiana stanu licznika na aktywny i następuje jego aktualizacja w sekundowych odstępach, a sekundy zamieniane są na format HH:mm:ss. Funkcja rozpocznij_prace() wywołuje metodę rozpocznijPrace() z klasy Pracownik, za pomocą której następuje zapis godziny rozpoczęcia pracy w bazie danych. Następnie wywoływana jest funkcja aktualizuj_licznik(), która uruchamia licznik. Podczas aktywności licznika, blokowana jest możliwość wylogowania się - zapobiega to powstawaniu wpisów bez godziny zakończenia pracy po jej zakończeniu. Funkcja zakoncz_prace() wywołuje metodę zakonczPrace() z klasy Pracownik, za pomocą której następuje zapis godziny zakończenia pracy w bazie danych. Następnie licznik jest zatrzymywany, a możliwość wylogowania się ponownie jest dostępna.

W skład panelu administratora wchodzi jedna funkcja - pokaz_raport():

```
def okno_administrator(administrator):
    wyczysc_okno()

    label_nazwa_administratora = ctk.CTkLabel(root, text=f"panel administratora:
{administrator.nazwa}", font=("Arial", 20))
    label_nazwa_administratora.pack(pady=(75, 10))

    lista = Listbox(root, width=90, height=15, font=("Arial", 12))
    lista.pack(pady=20)

    def pokaz_raport():
        lista.delete(0, END)

        raport = administrator.wyswietlRaporty()
        if len(raport) == 0:
            lista.insert(END, "brak danych!")
            return

        for pracownik in raport:
            pracownik_id = pracownik[0]
            nazwa = pracownik[1]
            email = pracownik[2]
            czasy_pracy = pracownik[3]

            lista.insert(END, f"pracownik: {nazwa}, e-mail: {email}")
            if len(czasy_pracy) == 0:
                lista.insert(END, "brak wpisow czasu pracy!")
            else:
                for czas_pracy in czasy_pracy:
                    rejestr = uzytkownik.RejestrCzasu(pracownik_id)
                    liczba_godzin = rejestr.obliczCzas(czas_pracy[0],
```

```

czas_pracy[1])
        lista.insert(END, f"start: {czas_pracy[0]}, koniec:
{czas_pracy[1]}, czas: {liczba_godzin}")
        lista.insert(END, "=====")

        button_raport = ctk.CTkButton(root, text="wyswietl raport",
command=pokaz_raport, **BUTTON)
        button_raport.pack(pady=15)

        button_wyloguj = ctk.CTkButton(root, text="wyloguj", command=okno_menu,
**BUTTON)
        button_wyloguj.pack(pady=15)

```

Powyższy kod usuwa wszystkie elementy poprzednio wyświetlanego okna, a następnie tworzy etykietę służącą do wyświetlania nazwy administratora oraz komponent Listbox, do którego wpisywane będą czasy pracy poszczególnych pracowników. Funkcja pokaz_raport() rozpoczyna swoje działanie od usunięcia z listy wcześniejszych wpisów. Następnie wywoływana jest metoda wyswietlRaporty() z klasy Administrator, odpowiedzialna za utworzenie raportu. Dla każdego pracownika pobierane są następujące dane: identyfikator, nazwa użytkownika, adres e-mail oraz godzina rozpoczęcia i zakończenia pracy. Dane te dodawane są do listy, a następnie obliczany jest łączny czas pracy. Dla każdego wpisu tworzony jest nowy obiekt uzytkownik.RejestrCzasu(), za pomocą którego wykonywane są obliczenia. W przypadku braku wpisów dotyczących pracowników oraz czasu ich pracy, wyświetlane są stosowne komunikaty.

13. Wnioski końcowe

System został poprawnie zaprojektowany i zaimplementowany w celu realizacji pierwotnego celu, czyli bezpiecznej rejestracji i monitorowania czasu pracy pracowników. Aplikacja umożliwia rejestrację użytkowników (z podziałem na role), logowanie, notowanie godzin rozpoczęcia i zakończenia pracy oraz generowanie raportów dla administratora. Wszystkie te funkcje działają zgodnie z opisem przypadków użycia i diagramów aktywności i pozostałych. Prosty, ale efektywny interfejs aplikacji ma wyraźne przyciski "Rozpocznij pracę" i "Zakończ pracę" eliminujące ryzyko pomyłek przy ręcznym wpisywaniu czasu. Metryki kodu pokazują, że liczba klas, metod i modułów jest optymalna do wielkości aplikacji i operacji na bazie danych.

14. Link do repozytorium

<https://github.com/szymonkuzio1/System-rejestracji-czasu-pracy>